



ECE PARIS • LYON
ÉCOLE D'INGÉNIEURS

Algorithmique et programmation structurée en C

Cours n° 2

Ordinateurs, environnement de travail et premiers pas en Langage C

Antoine Hintzy

d'après le cours
d'**Elisabeth Rendler**



Plan du cours

1. **Ordinateur**
2. **Bases arithmétiques (bases 10 et 2 (binaire))**
3. **Unités de mesure de quantité d'information numérique (octet)**
4. **Compilation**
5. **IDE : Environnement de Développement**
6. **Langage C**
7. **Bonnes pratiques d'écriture de code**

Modèle en couches de l'Ordinateur

Utilisateur·rice



Interfaces

Fenêtre, menus, aide contextuelle, assistants...

Logiciels et données

Navigateur, document.docx, projet ING1 en C...

Système d'exploitation

Gestion des périphériques, de la mémoire,
arbitrage des ressources...

Matériel

Processeur, mémoire vive, disque dur...

Ordinateur

Composants (matériel)

- Écran
- Clavier
- Souris / Trackpad
- Autres périphériques : imprimante, webcam, enceintes, joystick, disque dur externe, clé USB...
- **Unité centrale**

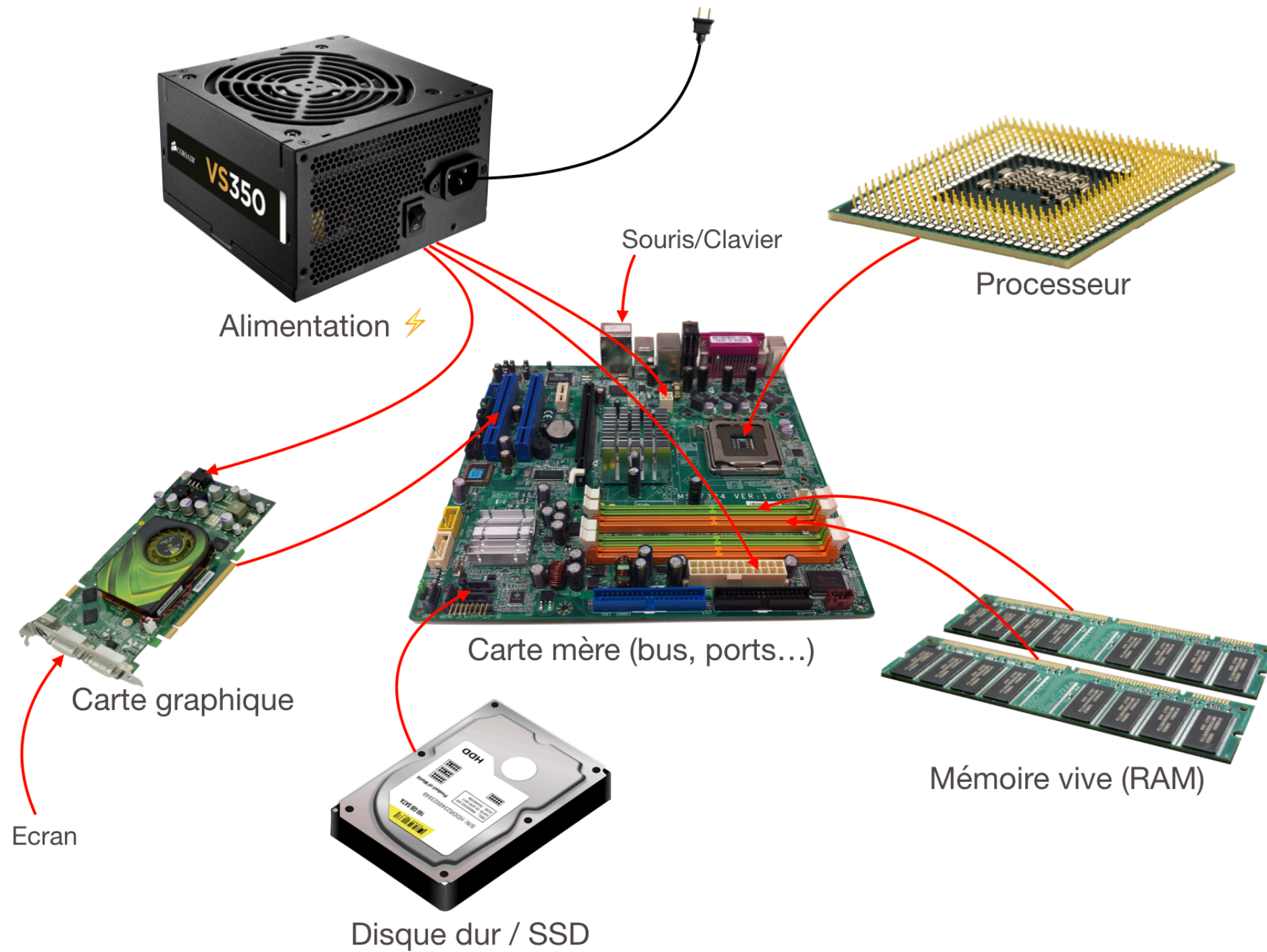
Unité centrale

De nos jours, il n'y a plus vraiment d'unité centrale, les composants sont souvent regroupés derrière l'écran  ou sous le clavier . Cependant, on y retrouve toujours les mêmes composants :

- Carte mère
- *Microprocesseur* (Intel, AMD...)
- Mémoire vive (RAM)
- Disque dur interne  (disque classique ou **SSD**)
- Bloc d'alimentation 
- Cartes graphique, son, réseau...

Unité centrale

Composants



Carte mère - Centralisation des composants 🤰

La carte mère est essentiellement composée de circuits imprimés, de bus et de ports de connexion, lui permettant d'assurer la **liaison entre tous les composants et périphériques** de l'ordinateur.

Au démarrage, le microprocesseur exécute le code contenu dans le **BIOS** (*Basic Input Output System*) ou l'**UEFI** (*Unified Extensible Firmware Interface*) de la carte mère afin de pouvoir reconnaître et configurer chaque périphérique qui y est connecté. Ils détiennent le "plan" de la carte mère.

Processeur - Traitement de l'information

Le processeur **traite les opérations** et instructions machine des programmes informatiques.

Tâches :

- Lire et écrire les données en mémoire
- Traiter les données (calculs)

Processeur - Traitement de l'information 🧠

On désigne également un processeur par les termes :

- ◎ 🇬🇧 *Central Processing Unit* (**CPU**)
- ◎ 🇫🇷 Unité Centrale de Traitement (**UCT**)

On parle de **Microprocesseur** s'il ne se compose que d'un seul circuit intégré (une seule carte).

Processeur - Traitement de l'information

Performances

La vitesse d'un processeur (**vitesse d'horloge**) se mesure en **Hertz** (Hz , plutôt en GHz d'ailleurs) et correspond au **nombre d'opérations pouvant être traitées par seconde** par le processeur.

Les nouvelles puces de processeurs contiennent aujourd'hui **plusieurs cores**, ils correspondent chacun à un processeur et communiquent entre eux pour augmenter la vitesse d'exécution des opérations.

Processeur - Traitement de l'information

Opérations

Les opérations sont **câblées dans le processeur** ; c'est-à-dire qu'elles sont effectuées par des **circuits électroniques**.

Les processeurs incorporent au moins l'addition, la soustraction et la multiplication.

La plupart des processeurs incorporent en plus la division, la racine carrée, des fonctions trigonométriques (cosinus) ou encore exponentielles... À défaut, elles doivent traiter les calculs complexes avec leurs propres opérations, de façon logicielle.

Les derniers processeurs intègrent de plus en plus de calculs câblés, ce qui les rend plus rapides.

Coprocesseurs

Les coprocesseurs sont comme des processeurs. Ils viennent en complément et sont spécialisés dans des tâches précises :

- **Carte graphique** : ajoute des circuits câblés permettant d'accélérer les rendus 2D et/ou 3D.
- **Carte arithmétique** : optimise les opérations sur des nombres à virgule flottante.
- ...

Mémoire centrale - Mémoire vive (RAM)

On appelle **mémoire centrale** l'endroit où sont stockées temporairement les données et les programmes en cours de traitement par le processeur.

Bien que les disques SSD occupent de plus en plus souvent ce rôle, la mémoire centrale est habituellement stockée en **mémoire vive (RAM)** :

- **Rapide** : fournit rapidement les données au processeur.
- **Volatile** : perte des données dès que l'ordinateur n'est plus alimenté en électricité (contrairement à la mémoire ROM, disques durs) ou lorsque le programme associé se termine.

Les valeurs des variables de nos programmes seront stockées en mémoire centrale.



Stockage de l'information

Disque dur / SSD / mémoire flash

Permettent de stocker des fichiers sur le long terme, au delà de la durée de vie d'un programme, même lorsque l'ordinateur est éteint.

Le disque dur stocke le système d'exploitation (**OS**), les programmes installés et les fichiers du/des utilisateur·s.

- **Disque dur** : stockage physique sur bande magnétique, sensible aux chocs, relativement lent.
- **Mémoire flash** : rapide, résistant aux chocs, prix élevé, nombre d'écritures (de moins en moins) limité.

*Exemples : clé USB, carte mémoire pour appareil photo numérique, **disque SSD**...*

Systeme d'Exploitation (SE)

Operating System (OS)

Le système d'exploitation est le programme qui assure la communication entre le matériel (la machine) ou les périphériques et les programmes qui définissent et appliquent le traitement des données. Il gère la mémoire, les autorisations (lecture/écriture), les périphériques (écran, clavier...), affectue l'arbitrage des ressources...

Exemples : Windows, MacOS, Linux.

Base 10 🙌🙌

Dans la vie de tous les jours, nous comptons en base 10.
Nos nombres sont une succession de chiffres allant de 0 à 9 :

$$\begin{aligned} 12 &= \dots 00012 \\ &= \dots 0 \qquad \qquad \qquad 0 \qquad \qquad \qquad 0 \qquad \qquad \qquad 1 \qquad \qquad \qquad 2 \\ &= \dots 0 \times 10^4 \qquad + 0 \times 10^3 \qquad + 0 \times 10^2 \qquad + 1 \times 10^1 \qquad + 2 \times 10^0 \\ &= \dots 0 \times 10000 \qquad + 0 \times 1000 \qquad + 0 \times 100 \qquad + 1 \times 10 \qquad + 2 \times 1 \\ &= \dots 0 \qquad + 0 \qquad + 0 \qquad + 10 \qquad + 2 \end{aligned}$$

Base 2 - Le binaire 🙌

En informatique, les données sont manipulées au format **binaire** (base 2).
Il s'agit d'une succession de 0 et de 1 (appelés des **bits** ( Binary Digits)).

$$\begin{aligned} 12 &= \dots 01100 \\ &= \dots 0 && 1 && 1 && 0 && 0 \\ &= \dots 0 \times 2^4 && + 1 \times 2^3 && + 1 \times 2^2 && + 0 \times 2^1 && + 0 \times 2^0 \\ &= \dots 0 \times 16 && + 1 \times 8 && + 1 \times 4 && + 0 \times 2 && + 0 \times 1 \\ &= \dots 0 && + 8 && + 4 && + 0 && + 0 \\ &= 12 \end{aligned}$$

Base 2 - Le binaire 🤞

Pourquoi ?

Le binaire est beaucoup plus simple à symboliser de façon physique :

- Électricité ⚡ : $0v$ pour le 0, $5v$ pour le 1.
- Fibre optique 💡 : pas de lumière pour le 0, lumière pour le 1.
- Bande magnétique 🔄 : orientation de l'aimantation.
- ...

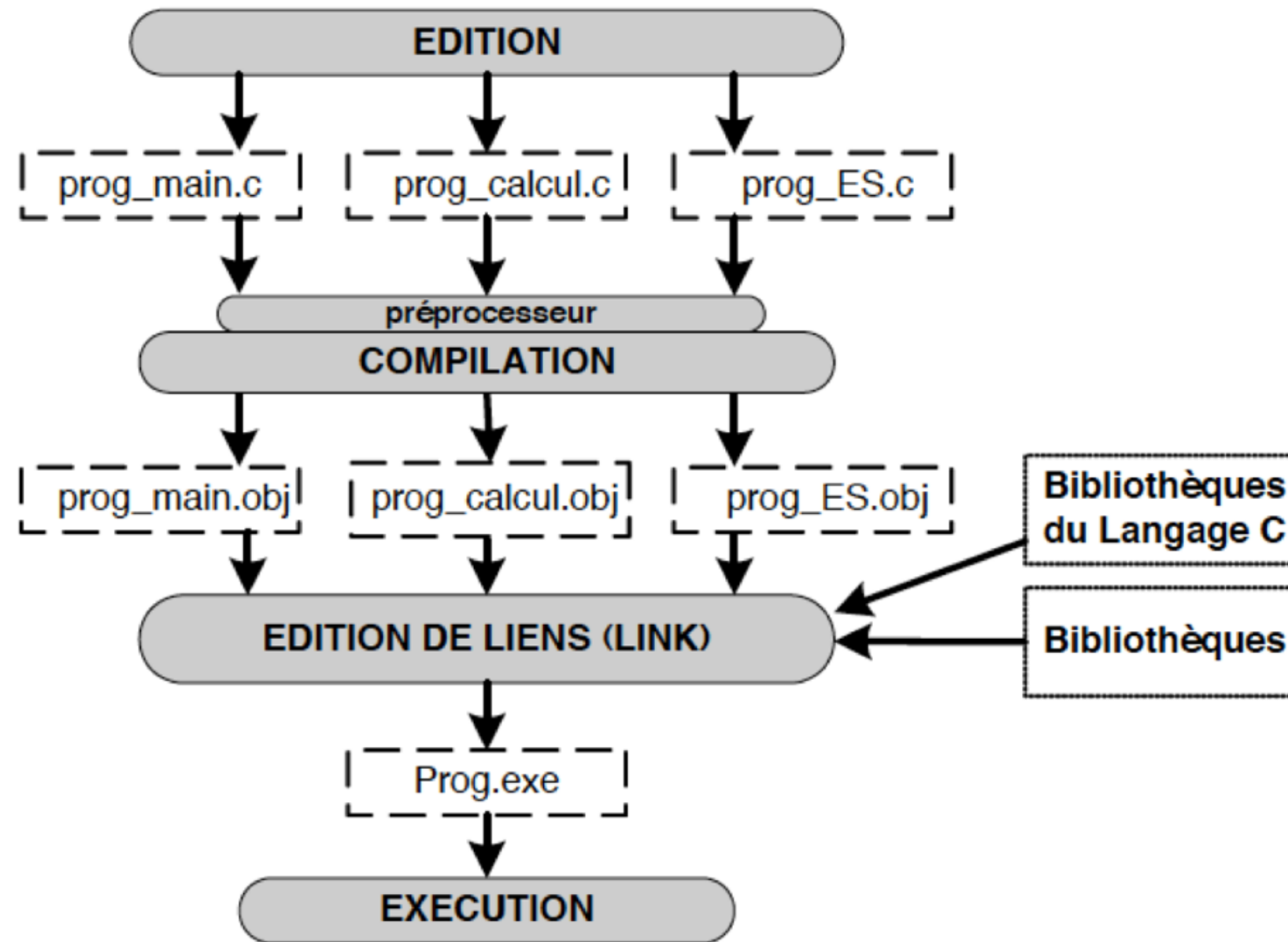
Mémoire

Unités de mesure de quantité d'information numérique

Unité	Taille
1 Octet (souvent confondu avec  Byte)	1o = 8 bits
1 Kilooctet (Ko /  Kb)	1Ko = 1 024 octets (2_{10} octets)
1 Mégaoctet (Mo /  Mb)	1Mo = 1 048 576 octets (2_{20} octets)
1 Gigaoctet (Go /  Gb)	1Go = 1 073 741 824 octets (2_{30} octets)
1 Téraoctet (To /  Tb)	1To = 1 099 511 627 776 octets (2_{40} octets)

NB : 1 Byte ne correspond en réalité pas à 1 octet, mais à 1 **multiplet** (plus petite unité de mémoire logiquement adressable, qui n'a pas toujours été de 8 bits). Un octet est donc en réalité un multipler/bye de 8 bits. Le nombre 8 s'étant généralisé, on utilise aujourd'hui communément byte pour désigner un octet.

Compilation



Source : http://public.iutenligne.net/informatique/algorithmes-et-programmation/priou/LangageC/101_fabrication_du_programme__compilation__prprocesseur.html

Environnement de développement (IDE^{🇬🇧}) - Code::Blocks

🇬🇧 IDE : *Integrated Development Environment*

🇫🇷 EDI : Environnement de Développement Intégré (*peu utilisé*)

Un *IDE* est un logiciel regroupant un ensemble d'outils visant à augmenter la productivité des développeurs dans leur travail. Les IDE intègrent notamment :

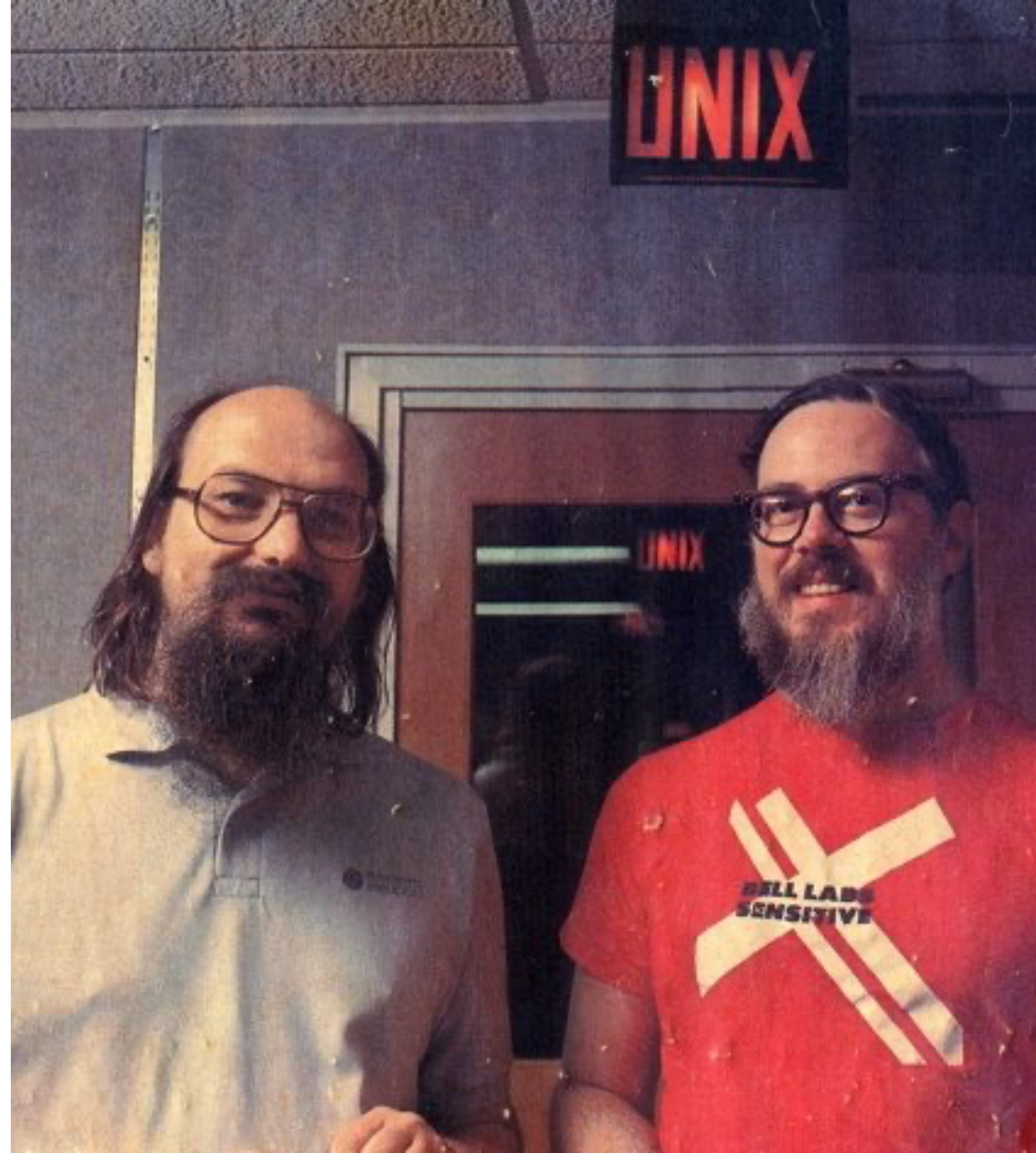
- Un éditeur de texte amélioré (*assistance, auto-completion...*).
- Un débogueur permettant d'exécuter le programme ligne par ligne.
- Une aide à la compilation et à l'édition des liens.
- ...

Le langage C

Le langage C a été inventé au début des années 70 dans les **Laboratoires Bell**, pour réécrire **UNIX** (noyau de la quasi-totalité des *OS* : Linux, MacOS, iOS... (sauf Windows)).

Il a été créé par **Dennis Ritchie** (à droite sur la photo), à partir du langage B initialement créé par Ken Thompson (à gauche).

Source : Wikipedia.



Le langage C

Le C est un langage de programmation **impératif** (séquence d'instructions) généraliste, de **bas niveau** (très proche de la machine, donc puissant, mais lourd à prendre en main).

Sa syntaxe et sa logique ont été réutilisées dans la majorité des langages de programmations modernes :
C++, C#, Java, PHP, Javascript...

Le C est encore très utilisé aujourd'hui, car il offre un grand contrôle sur la machine (la mémoire notamment).

Il est également utilisé en tant que fondation des langages modernes (compilateur/interpréteur) et systèmes d'exploitations.

Exemple de programme en C

```
// on inclue la bibliothèque qui permet d'afficher un message dans la console :
#include <stdio.h>

// programme principal (point d'entrée de notre logiciel) :
int main(void)
{
    // création d'une variable initialisée à 0 :
    int compteurEntier = 0;

    // boucle allant de 0 à 9 inclus (la valeur est stockée dans "compteurEntier") :
    for(compteurEntier = 0; compteurEntier < 10; compteurEntier++)
    {
        // on affiche la valeur dans la console :
        printf("La variable compteurEntier vaut %d", compteurEntier);
    }

    return 0;
}
```

Découvrons quelques bases du langage C

- Fichiers C
- Code de base d'un programme en C
- Les commentaires
- Les variables (et les types scalaires)
- Transtypage (*cast*)
- Les constantes (*const*) et macros (*#define*)
- Les opérateurs (+, -, /...)

Fichiers C

En C, les programmes s'écrivent dans un ou plusieurs fichiers texte d'**extension .c**.

Ils contiennent le **code source** de notre programme.

Le **fichier principal** (de départ) d'un programme s'appelle **main.c**.

Code de base d'un programme en C

Le fichier **main.c** contient au minimum :

```
int main()  
{  
  
    return 0;  
}
```

L'instruction **return 0;** de cette fonction **main** représente la **FIN** de notre programme. Tout ce qui se trouve après est ignoré.
Elle signale au système que le programme s'est terminé correctement.

Les commentaires

Ajoutent des **indications à destination des développeurs**, qui seront ignorées au moment de la compilation. En C, est considéré comme commentaire tout ce qui se trouve :

- à droite de deux slashes (`//`), jusqu'à la fin de la ligne.
- entre `/*` et `*/` (commentaires mono ou multilignes).

Exemples :

```
/* Nous allons maintenant déclarer deux  
   variables i et j, initialisées à 0 : */
```

```
int i = 0; // Déclaration de la variable i, initialisée à 0
```

```
int j = 0; /* Déclaration de la variable j, initialisée à 0 */
```

Les variables

Une variable sert à **stocker une donnée en mémoire centrale**, afin de pouvoir la traiter dans un programme.

Comme son nom l'indique, **la valeur d'une variable peut varier** au cours du temps.

En C, chaque variable possède un **type**, définissant le format de la donnée, ainsi que sa taille en mémoire.

(permet au système de savoir **combien d'octets** utiliser pour la stocker (1 octet pour un caractère contre 2 ou 4 octets pour un entier)).

Le type d'une variable ne peut pas changer une fois défini.

Les variables - Les types scalaires

En C, il existe deux familles de **types** de base (scalaires) :

- Les types **entiers** :

- **int, short, long, long long, char.**

- (les caractères sont considérés comme des entiers (voir table **ASCII**)).*

- Les types **réels** :

- **double, float** (nombre flottant).

NB : il n'existe normalement pas de type **Booléen** en C, nous stockons alors *vrai* et *faux* dans un entier (**int**) valant **1** (ou toute autre valeur non nulle) ou **0** . En pratique, les compilateurs récents comprennent le type **bool** (C++) pouvant avoir comme valeur **true** ou **false**.

Les variables entières - Bit de signe

Les types entiers peuvent **dédier un bit au signe (+/-)**. Par défaut, les types entiers sont signés, cela veut dire qu'ils consacrent un bit pour le signe.

Les types entiers peuvent alors être précédés du modificateur **unsigned** ou **signed** (par défaut) lors de leur déclaration.

Le modificateur **unsigned** permet de gagner un bit pour les valeurs (en perdant la notion de signe), permettant ainsi de stocker des nombres positifs 2 fois plus grands.

Considérons que sur notre machine, les entiers (**int**) sont stockés sur 4 octets (32 bits) :

- **int** ou **signed int** : valeur comprise entre $-2\,147\,483\,648$ (-2^{31}) et $+2\,147\,483\,647$ ($2^{31}-1$).
- **unsigned int** : valeur comprise entre 0 et $4\,294\,967\,295$ ($2^{32}-1$).

Les variables - Les types scalaires - Taille en mémoire

Type	Taille en mémoire	Borne inférieure (signed)	Borne supérieure (signed)	Borne inférieure (unsigned)	Borne supérieure (unsigned)	Initialisation en C
char	1 octet ($2^1 \times 8 = 2^8 = 256$ possibilités)	-128 (-2^7)	+127 (2^7-1)	0	255 (2^8-1)	char c = 'A'; char c2 = 65;
short (ou short int)	2 octets ($2^2 \times 8 = 2^{16}$ possibilités)	-32 768 (-2^{15})	+32 767 ($2^{15}-1$)	0	65 535 ($2^{16}-1$)	short s = 0;
int	2 ou 4 octets ($2^4 \times 8 = 2^{32}$ possibilités (si 4 octets))	-2 147 483 648 (-2^{31})	+2 147 483 647 ($2^{31}-1$)	0	4 294 967 295 ($2^{32}-1$)	int i = 0;
long (ou long int)	8 octets ($2^8 \times 8 = 2^{64}$ possibilités)	-9 223 372 036 854 775 808 (-2^{63})	+9 223 372 036 854 775 807 ($2^{63}-1$)	0	+18 446 744 073 709 551 615 ($2^{64}-1$)	long l = 0;
float	4 octets	1.2E-38	3.4E+38	--	--	float f = 0.0f;
double	8 octets	2.3E-308	1.7E+308	--	--	double d = 0.0;
long double	10 octets	3.4E-4932	1.1E+4932	--	--	double d = 0.0;

Les variables - Nommage

En C, les noms de variables se construisent avec :

- des **lettres** non accentuées, majuscules ou minuscules
- des **chiffres** (sauf le premier caractère)
- des ***underscores*** (tiret du bas)

Par convention, les noms de variables commencent toujours par une lettre minuscule.

Seuls les noms de structures (que nous verrons plus tard) pourront commencer par une majuscule.

Le nom des variables est très important, il ne faut pas hésiter à mettre un nom un peu long, mais explicite.

L'IDE nous aidera à aller vite sans réécrire le nom entier à chaque fois.

Bonne pratique : Utiliser la notation ***Camel Case*** : tout en minuscules, sauf le début des nouveaux mots :
exemple : **maNouvelleVariableBienNommee**.

Les variables - Déclaration

La déclaration d'une variable vise à **la faire connaître du système**, afin qu'il lui **alloue un espace en mémoire centrale**.

Elle se fait obligatoirement **avant toute utilisation de celle-ci, dans un bloc*** (idéalement au tout début du bloc, juste après l'accolade ouvrante). Pour déclarer une variable, on indique :

- son **type**

Qui déterminera les valeurs qu'elle peut prendre, mais également sa taille en mémoire.

- suivi de son **nom**

Un nom de variable ne peut être attribué qu'une seule fois par bloc (entre deux { accolades }).

⚠ Lors de la déclaration d'une variable, le système lui réserve un espace mémoire qui possède déjà des bits à 0 ou à 1.
Il est donc primordial de **toujours initialiser ses variables !**

* La déclaration d'une variable en dehors d'un bloc crée une **variable globale** (déconseillé).

Les variables - Déclaration

Exemple :

```
int main()
{
    // Déclarations au début du bloc (après l'accolade ouvrante) :
    char groupe;
    unsigned int age;
    float moyenneEnMathematiques;
    int etage, nbEtages;           // il est possible de déclarer plusieurs variables
                                   // de même type en les séparant par une virgule.

    // code...

    return 0;
}
```

Bonne pratique : Pensez à initialiser les variables en leur donnant une valeur au moment de leur déclaration.

Les variables - Affectation

L'affectation d'une valeur à une variable consiste à lui donner cette valeur. Cette opération se fait avec l'opérateur '=', suivi d'une valeur ou de quelque chose retournant une valeur (fonction, autre variable possédant déjà une valeur...).

Exemples :

```
int main() {  
    int maVariable, maVariable2, maVariable3; // déclarations obligatoire avant utilisation  
  
    // affectations :  
    maVariable = 15; // maVariable prend la valeur 15  
    maVariable2 = maVariable + 10; // maVariable2 prend la valeur de maVariable + 10 (donc 25)  
    maVariable3 = maVariable; // maVariable3 prend la valeur de maVariable (donc 15)  
    maVariable3 = 10 * 4; // maVariable3 prend la valeur de 10 * 4 (donc 40)  
}
```

Les variables - Affectation

Affectations cachées : opérateurs `++`, `--`, `*=`, `+=`, `-=`...

```
int i = 0; // déclaration + affectation = initialisation
```

```
i++; // semblable à i = i + 1; (puis "retourne" l'ancienne valeur)
```

```
++i; // semblable à i = i + 1; (puis "retourne" la nouvelle valeur)
```

```
i--; // semblable à i = i - 1; (puis "retourne" l'ancienne valeur)
```

```
--i; // semblable à i = i - 1; (puis "retourne" la nouvelle valeur)
```

```
i *= 2; // semblable à i = i * 2;
```

```
// etc.
```

Les variables - Initialisation

Une initialisation est une **déclaration combinée à une affectation**, donnant sa première valeur à la variable.

✓ Il est conseillé de **toujours initialiser ses variables !**

Exemple :

```
int entier1 = 10,  
    entier2 = 0;
```

NB : une variable non initialisée hérite de la valeur déjà présente dans l'espace mémoire qui lui est alloué par le système lors de sa déclaration.

Les variables - Transtypage (cast)

Il peut arriver que l'on veuille **convertir la valeur d'une variable dans un autre type**.

On appelle cela un  **cast** :

```
/** Cast explicite : */  
int var1 = 0;  
var1 = (int) 8.456; // cast explicite d'un réel en entier, en spécifiant  
                  // le type souhaité entre parenthèses. var1 vaut maintenant 8.
```

```
/** Cast implicite : */  
int var2 = 0;  
var2 = 'A'; // var2 étant de type entier (int), le caractère 'A' (de type char) est  
            // implicitement casté en entier, prenant ainsi la valeur entière  
            // correspondante à 'A' dans la table ASCII : 65 (voir table ASCII).
```

```
float x1 = 0; // cast implicite de l'entier 0 au float 0.0f.  
float x2 = 0.0; // cast implicite du double 0.0 au float 0.0f.  
float x3 = 0.0f; // pas de cast !
```

Les variables - Transtypage (cast)

Exemple

```
#include <stdio.h>
```

```
int main(void) {  
    int a = 5,  
        b = 2;
```

```
    float sansCast = 5 / 2;  
    float avecCast = (float) 5 / (float) 2;
```

```
    printf("Sans cast : %f\n", sansCast); // Affiche : "Sans cast : 2.000000"  
    printf("Avec cast : %f\n", avecCast); // Affiche : "Avec cast : 2.500000"
```

```
    return 0;  
}
```

Les variables - Les caractères (char)

Les caractères sont considérés comme des **entiers**.

À partir d'un nombre entier, l'ordinateur peut savoir de quel caractère il s'agit d'après la **table ASCII**.

American Standard Code for Information Interchange

Par exemple, l'entier **65** désigne le caractère '**A**', l'entier **66** désigne '**B**' ...

⚠ Les caractères sont notés entre quotes simples : '**3**', ..., '**A**', '**B**', ..., '**a**', '**b**'.

⚠ Attention à ne pas confondre **char c = 0;** et **char c = '0';**.

✗ La notation "**a**", ou encore "**ab**" (avec des guillemets doubles), représente une chaîne de caractères (nous verrons ça plus tard).

```
// Les trois lignes suivantes sont identiques :  
char unCaractere = (char) 65; // cast explicite  
char unCaractere = 65;        // cast implicite  
char unCaractere = 'A';
```

Table ASCII (American Standard Code for Information Interchange)

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Source : https://fr.wikipedia.org/wiki/American_Standard_Code_for_Information_Interchange

Table ASCII

Exercice

Comment afficher l'alphabet ?

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Table ASCII

Exercice

Comment afficher l'alphabet ?

✗ Pas idéal... :

```
printf("A\n");
printf("B\n");
printf("C\n");
// ...
printf("Z\n");
```

ou

```
printf("A\nB\nC\n...\nZ\n");
```

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Table ASCII

Exercice

Comment afficher l'alphabet ?

✓ Mieux :

```
int i = 0;

for(i = 65; i < 65+26; i++) {
    printf("%c\n", (char) i);
}
```

✓ Beaucoup mieux :

```
char c = 0;

for(c = 'A'; c <= 'Z'; c++) {
    printf("%c\n", c);
}
```

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	\$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[ENG OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Les constantes (const)

Les constantes sont des variables dont **la valeur ne peut pas changer**.

Elles seront surtout utiles lorsque nous créerons des fonctions.

Pour en créer une il suffit de rajouter le **modificateur const** avant le type de la variable, au moment de sa déclaration.

Exemple :

```
const int cetteVariableVaudraToujours5 = 5;
```

Les constantes sont de vraies variables, cela veut dire que le compilateur va les compiler, et pourra ainsi détecter les éventuelles erreurs de type (si on passe un entier à une fonction alors qu'elle attend un nombre décimal), ce qui n'est pas possible avec les macros (slide suivante).

Les macros - constantes de préprocesseur (`#define`)

Les macros sont des **rechercher/remplacer** effectués par le préprocesseur, juste **avant la compilation**.

Elles sont principalement utilisées afin de créer des **constantes de paramétrage** qui permettent de réaliser des modifications à plusieurs endroits dans le code en ne modifiant qu'un seul endroit.

Par convention, les macros sont nommées en **lettres capitales**, avec un ***underscore*** entre chaque mot :

```
#define AGE_MINIMUM 12
```

Contrairement aux constantes classiques, les macros ne prennent pas de place en mémoire centrale, puis le rechercher/remplacer est effectué avant la compilation, la valeur est donc stockée directement dans l'exécutable. L'inconvénient est l'absence de vérification au moment de la compilation, mais ne permet pas une vérification de type par le compilateur. **NB :** Pas de point virgule, sinon le point virgule sera copié dans le rechercher/remplacer.

Les macros - Pourquoi ?

Prenons l'exemple suivant :

```
int main()
{
    int ageUtilisateur = 12; // dans un vrai programme, l'âge serait demandé à l'utilisateur !
    if (ageUtilisateur < 12)
    {
        printf("Vous ne pouvez pas regarder ce film.");
    }
    else if (ageUtilisateur == 12)
    {
        printf("Vous pouvez enfin regarder ce film.")
    }
    return 0;
}
```

Imaginons que nous souhaitons changer l'âge limite pour visionner le film. Il suffit en théorie de remplacer **12** par le nouvel âge limite. Ici, c'est facile car il n'y a que 3 fois le nombre **12**, on voit rapidement qu'il ne faut changer que les 2 dernières occurrences. Mais dans un vrai programme, il peut y en avoir beaucoup plus, et dans plusieurs fichiers. Les constantes permettent donc un paramétrage plus facile (voir slide suivante).

Les macros - Pourquoi ?

Puisque l'âge limite est une donnée de paramétrage, on préfère le transformer en macro :

```
#define AGE_LIMITE 12

int main()
{
    int ageUtilisateur = 12; // dans un vrai programme, l'âge serait demandé à l'utilisateur !
    if (ageUtilisateur < AGE_LIMITE)
    {
        printf("Vous ne pouvez pas regarder ce film.");
    }
    else if (ageUtilisateur == AGE_LIMITE)
    {
        printf("Vous pouvez enfin regarder ce film.")
    }
    return 0;
}
```

Si on décide de changer l'âge limite, il n'y a alors plus qu'un seul endroit à modifier !
Donc aucune confusion possible avec les autres **12** du code source.

Instructions de base - Les opérateurs

Les opérateurs s'utilisent entre deux opérandes qui sont soit une valeur (**3**), soit quelque chose retournant une valeur : une variable (**var1**) stockant une valeur, une fonction...

On peut choisir de placer ou non un espace entre les opérandes et leur opérateur. Il est toutefois conseillé d'être constant dans sa décision :

```
int somme = 0;
```

```
somme = 3 + 6; // la variable est déclarée (initialisée) donc on peut l'utiliser  
// ou  
somme=3+6;
```

Les opérateurs mathématiques

Opérateur	Symbole en C	Exemple	Résultat de l'exemple
Addition	+	3 + 2	5
Soustraction	−	3 − 2	1
Multiplication	*	3 * 2	6
Division	/	5 / 2 (division de deux entiers)	2 (le résultat est un entier)
Modulo (reste de la division)	%	5 % 2	1

Priorité des opérateurs

Il existe de nombreux autres opérateurs (binaires par exemple). Nous en verrons certains d'entre eux plus tard. Voici la table des priorités des opérateurs en C :

opérateur	associativité
() [] -> .	de gauche à droite
! - ++ -- ~ (type) * & sizeof	de droite à gauche
* / %	de gauche à droite
+ -	de gauche à droite
<< >>	de gauche à droite
< <= > >=	de gauche à droite
= = !=	de gauche à droite
&	de gauche à droite
^	de gauche à droite
	de gauche à droite
&&	de gauche à droite
	de gauche à droite
? :	de droite à gauche
= += -= *= /= >>= <<= &= ^= =	de droite à gauche
,	de gauche à droite

Entrées/sorties console

Pour interagir avec la console, nous avons besoin d'une **bibliothèque standard** : **stdio.h**.

Les **bibliothèques** contiennent des fonctions (outils) utilisables par tous les programmes, une fois importées. Il existe des bibliothèques pour interagir avec la console, pour créer des fenêtres (Allegro par exemple), pour se connecter à une base de données...

Pour utiliser une bibliothèque standard, il suffit de l'inclure là où l'on souhaite l'utiliser, avec l'instruction de préprocesseur **#include** (tout en haut du fichier) :

```
#include <stdio.h>
```

Affichage à l'écran - ÉCRIRE "..."

```
printf(chaineAffichee, variablesEventuelles);
```

// Exemple :

```
int n = 3;
```

```
printf("La valeur de la variable n est :\t%d\n", n);
```

`\n` représente un **retour à la ligne**, `\t` une **tabulation**.

`%d` indique que l'on souhaite insérer la valeur d'un entier dans la chaîne. L'entier en question se trouvant après le texte entre guillemets (**n**).

Autant de valeurs/variables sont attendues à droite du texte entre guillemets qu'il y a de `%...` dans le texte : **`printf("n = %d et n au carré = %d", n, n*n);`**

Affichage à l'écran - Format des données

Type	Format dans le texte du printf
Entier	%d
Réel	%f
Caractère	%c
Entier/Réel long	%ld / %lf
Chaîne de caractères	%s
Hexadécimal	%x

Lecture au clavier - LIRE n

```
scanf(format, adresseVariable);
```

```
// Exemple :
```

```
int n = 0;
```

```
scanf("%d", &n);
```

L'esperluette (&) permet de récupérer l'**adresse mémoire** de la variable **n** afin d'y écrire la valeur lue au clavier. Attention à ne pas l'oublier en utilisant **scanf**.

Exemple de programme C

```
#include <stdio.h>

int main() { // DEBUT
    // Initialisation des variables (avant manipulation) :
    int maVariable = 0;
    int y = 0;

    printf("Saisissez une valeur :\n");
    scanf("%d", &maVariable); // l'utilisateur tape 2 au clavier, maVariable vaut 2
    printf("maVariable vaut %d\n", maVariable++); // affiche : "maVariable vaut 2"
    // maVariable vaut 3
    printf("maVariable vaut %d\n", ++maVariable); // affiche : "maVariable vaut 4"
    // maVariable vaut 4
    maVariable /= 2; // maVariable vaut 2
    y = maVariable + 10; // y vaut 12, et maVariable vaut toujours 2

    return 0; // FIN
}
```

Bonnes pratiques d'écriture de code 😊

Indentation

On **indente** (= *décale vers la droite*) de x espace·s ou tabulation·s (*à définir*) tout le code se trouvant dans un **sous bloc** (délimité par une **paire d'accolades**).

```
int main(void)
{
    // tout ce qui se trouve entre les accolades (dans le bloc) est indenté d'un niveau supplémentaire
    int compteurEntier = 0;

    for(compteurEntier = 0; compteurEntier < 10; compteurEntier++)
    {
        // tout le code à l'intérieur de la boucle "for" (dans le sous bloc délimité
        // par ses accolades) est indenté d'un niveau supplémentaire vers la droite
        printf("La variable compteurEntier vaut %d", compteurEntier);
        // ainsi, on voit clairement quelles lignes sont répétées
    }

    return 0;
}
```

Bonnes pratiques d'écriture de code 😊

Nommage des variables

N'ayez pas peur de donner de longs noms explicites à vos variables, l'IDE vous aidera lors de leur écriture pour aller plus vite.

Mettez en place vos propres conventions de nommage :

- "s" à la fin des variables représentant plusieurs données (tableaux)
- Une seule langue : français 🇫🇷 / anglais 🇬🇧...
- ...

Bonnes pratiques d'écriture de code 😊

Casse

Nommez vos variables en utilisant une **casse** spécifique :

- **Camel case** 🐪 : `maVariableEntiere`

Pour les variables.

- **Pascal case** : `MaVariableEntiere`

Pour les noms de structures, ou les classes en C++.

- **Kebab case** 🥗 ou **Spinal case** 🦴 : `ma-variable-entiere`

Impossible en C.

- **Underscore case** ou **Snake case** 🐍 : `ma_variable_entiere` ou `MA_VARIABLE_ENTIERE`

Pour les constantes (lettres capitales).

Bonnes pratiques d'écriture de code 😊

Initialisez toutes les variables lors de leur déclaration

Une bonne pratique consiste à **initialiser toutes les variables avant toute instruction.**

```
int main(void) {  
    // Initialisations  
    int ageUtilisateur = 0, tailleUtilisateurEnCm = 0;  
  
    // Quand toutes les variables sont déclarées, on peut commencer les instructions  
    printf("Quel âge avez-vous ?");  
    scanf("%d", &ageUtilisateur);  
  
    return 0;  
}
```

Bonnes pratiques d'écriture de code 😊

Constance d'écriture, régularité

Adoptez un style d'écriture constant.

Par exemple, si vous décidez de mettre des espaces autour des opérateurs, faites-le toujours.

Bonnes pratiques d'écriture de code 😊

Commentaires

Commentez votre code lorsque cela est utile à la compréhension.

NB : un code avec des variables bien nommées ne nécessite pas de commentaire.

Vous pouvez également utiliser les commentaires pour faire des séparateurs décoratifs :

```
/******  
*   Déclaration des variables   *  
*****/  
unsigned compteur = 0;
```